

Relatório

Implementação do Scanner usando JFlex

Alunos:

- Andreas Muzzi
- Marco de Sousa Melo

1) IDE (Integrated Development Environment)

Utilizamos o Netbeans por estarmos mais familiarizados com sua interface.

2) Códigos em java (calculadora e LP selecionada)

<https://github.com/andreamzz/CompiladorCminus>

3) Informações da LP escolhida

Escolhemos **C minus** como linguagem de programação.

Sua sintaxe e descrição é encontrada na seção [Appendix A: A COMPILER PROJECT](#) do livro [Compiler Construction: Principles and Practice](#).

E como complemento, buscamos guias como o [“A Comprehensive Guide to the Cminus Programming Language”](#) que está disponível na web.

Essa linguagem é uma versão super simplificada de C, feita para disciplinas de compiladores. Suas características principais são:

- Suporta 2 tipos básicos (int e float) que podem formar vetores unidimensionais.
- É fortemente tipada (o tipo de cada expressão deve ser determinado em tempo de compilação)
- Não há booleanos nativos (operadores relacionais retornam inteiros 0 ou não 0).
 - Logo **-1** seria true.
- Identificadores começam com letra e só podem conter letras e números. E, são case sensitive.
 - É possível determinar um limite no tamanho dos identificadores, desde que pelo menos 31 caracteres (não colocamos limite).
- Palavras chaves reservadas:
 - else, exit, float, if, int, read, return, while, write.
- Símbolos e operadores:
 - AND: "&&"
 - ASSIGN: "="
 - CM: ";"
 - DIVIDE: "/"
 - EQ: "=="
 - GE: ">="

- GT: ">"
- LBR: "{"
- LBK: "["
- LE: "<="
- LP: "("
- LT: "<"
- MINUS: "-"
- NE: "!="
- NOT: "!"
- OR: "||"
- PLUS: "+"
- RBR: "}"
- RBK: "]"
- RP: ")"
- SC: ","
- SQ: ""
- TIMES: "*"
 - Comentários:
 - bloco /* ... */ pode aparecer antes ou depois de qualquer token e não aninham.
 - Estruturas de controle:
 - **if** (com **else**), **while** e blocos compostos { ... }.
 - Funções:
 - As funções não tem parâmetros, com retorno do valor da primeira instrução **return** executada.
 - Entrada, saída e término:
 - **read(x)**, **write(expr)**, **write('string')**, **exit**.
 - Em atribuição, o valor é convertido para o tipo da variável na esquerda.
 - Em uma operação que um **int** encontra um **float**, o **int** vira **float** antes da operação.
 - Ou seja: float a = 1 + 2.8; // 1 -> 1.0; resultado 3.8 (float)
 - Precedência (do maior pro menor):
 - *, / **é maior que** +, - **é maior que** relacionais **é maior que** ! **é maior que** ||, &&
 - relacionais:
 - ==, !=, <, <=, >, >=

Algumas diferenças relevantes ao comparar C com C minus:

- Não há ponteiros, char, structs enums, typedef, for, do..while, break/continue, switch, sizeof.
- Não há void.
- Não há comentário com //.
- Não existe v[i][j] (array com 2 dimensões).
- Não existe string como em C (char nome[2] = "oi").

4) Subconjunto de regras:

Para simplificação foram escolhidas as regras de expressão pois não envolvem palavras reservadas .

- ER de expressão

Assignment ::= IDENTIFIER ASSIGN Expr SC

Expr ::= Expr PLUS Expr
| Expr MINUS Expr
| Expr TIMES Expr
| Expr DIVIDE Expr
| IDENTIFIER
| INTCON
| FLOATCON
| LP Expr RP

- DFA
O DFA foi feito para identificar os tokens entrada
[link do DFA](#)
- Entrada
A entrada consiste de uma simples expressão com as 4 operações básicas
 $a = (a + 1 * (b + 2)) / 3$
[link da entrada](#)
- Resultados
O resultado a partir da DFA foi:
IDENTIFIER, ASSIGN, LP, IDENTIFIER, PLUS, INTCON, TIMES, LP, IDENTIFIER, PLUS,
INTCON, RP, RP, DIVIDE, INTCON
[link do resultado gerado pelo analisador léxico](#)

5) Decisões tomadas no processo de geração e execução do Scanner e dificuldades:

Como decidimos organizar os arquivos:

Eles estão na pasta "/c_minus".

- [scanner.flex](#):
 - Especificação JFlex.
 - Aqui definimos os regex, ordem das regras e o tratamento de brancos e erros.
- [GeraResultados.java](#):
 - Para facilitar os testes foi criada a classe que lê os arquivos de teste e escreve o resultado na pasta de saída.
- [parser.cup](#):
 - O parser ainda está em estado inicial.
- [Parser.java](#):
 - O parser ainda está em estado inicial.

- [Scanner.java](#):
 - Arquivo gerado pelo jFlex usando as definições feitas em CMinusLexer.flex.
- [Tokens.java](#):
 - Arquivo gerado pelo java-cup usando as definições feitas parser.cup.

Entradas de teste estão na pasta "/entrada":

- [1.txt](#) (esse tem erros propositalmente)
 - Avaliamos int, float, return e {}. Colocamos um erro proposital ~~. Logo, o scanner deve reportar erro léxico nesses 2 caracteres.
- [2.txt](#)
 - Aqui testamos expressões aritméticas e precedência: * é maior que +, -
- [3.txt](#)
 - Aqui avaliamos o controle de fluxo com if/else e relacional >
- [4.txt](#) (esse tem erros propositalmente)
 - Teste simples fora de função.
 - Não deve apresentar erro léxico, mas no parser deve apresentar erro por não ter declaração prévia.
- [5.txt](#) (esse tem erros propositalmente)
 - Teste de array unidimensional e de caractere inválido no fim.
- [6.txt](#) (esse tem erros propositalmente)
 - não deve apresentar nenhum erro léxico, pois a falta de parêntese é um erro sintático.
- [7.txt](#)
 - Expressão simples.

Demais decisões tomadas:

- As tomadas de decisões realizadas até aqui tiveram como principal objetivo simplificar o processo de desenvolvimento do compilador, já que estamos apenas no início do projeto.
- Optamos por implementar estruturas mais básicas e utilizar ferramentas que auxiliam na análise léxica e sintática, evitando complexidade desnecessária nesta fase inicial.
- Decidimos adicionar maior precedência para casos de dupla negação !!a == a **precedence right NOT**.

Dificuldades:

- Não achamos muitos documentos sobre o C minus, além disso os que tinham estavam em inglês.
- No início achamos que a posição das regras de ** e // em relação a * e / poderiam causar tokenização errada (dividir ** em * e *). Verificamos na documentação do JFlex e confirmamos que ele aplica maximal munch (pega o lexema mais longo) e só usa a ordem das regras em empates. Como ** e // são mais longos que * e /, não havia bug.
- Tivemos dificuldades com a escolha dos tokens escolhidos para DFA reduzida.
- Tivemos dúvidas na construção da DFA em casos com palavras reservadas então decidimos não incluí-las nessa etapa.

